Record Book Format:

Left Side	**Right Side**
Sample Input **Sample Output**	**Aim** **Algorithm** **Program** **Result**

1. Managing Completed Tasks in a Project Management System- Linked List

---

Aim:

To develop a Java program that deletes a specified number of completed tasks from the end of a task list and displays the remaining tasks.

Algorithm:

Step 1: Start the program.
Step 2: Read an integer N, representing the number of task IDs.
Step 3: Read N task IDs and store them in a linked list.
Step 4: Read an integer M, representing the number of completed tasks to be deleted from the end.
Step 5: Use a loop to remove the last element from the list M times or until the list becomes empty.
Step 6: If the list is empty, display “List is empty”.
Step 7: Otherwise, print the remaining task IDs separated by spaces.
Step 8: Stop the program.

Program Code:

```
import java.util.*;

class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class Deleteatback {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        // Create linked list
        Node head = null, tail = null;

        // Insert nodes manually
        for (int i = 0; i < n; i++) {
            int value = sc.nextInt();
            Node newNode = new Node(value);
            if (head == null) {
                head = newNode;
                tail = newNode;
            } else {
                tail.next = newNode;
                tail = newNode;
            }
        }

        int m = sc.nextInt();

        // Delete m nodes from the end
        for (int i = 0; i < m && head != null; i++) {
            head = deleteLastNode(head);
        }
```

```
        // Print result
        if (head == null) {
            System.out.println("List is empty");
        } else {
            printList(head);
        }

        sc.close();
    }

    // Function to delete last node
    static Node deleteLastNode(Node head) {
        if (head == null || head.next == null) {
            return null;
        }
        Node temp = head;
        while (temp.next.next != null) {
            temp = temp.next;
        }
        temp.next = null; // remove last node
        return head;
    }

    // Function to print the linked list
    static void printList(Node head) {
        Node temp = head;
        while (temp != null) {
            System.out.print(temp.data + " ");
            temp = temp.next;
        }
    }
}
```

Sample Input:

```
6
201 202 203 204 205 206
4
```

Sample Output:

201 202

Result:
The program successfully removes the specified number of completed tasks from the end of the list and displays the remaining active task IDs.

2. Adding a Task at a Specific Position- Linked List

Aim:

To develop a Java program that inserts a new task (represented by a unique task ID) at a specific position in a linked list and displays the updated task list.

Algorithm:

Step 1: Start the program.
Step 2: Read an integer N, the number of task IDs initially present in the list.
Step 3: Read N space-separated task IDs and create a linked list using nodes.
Step 4: Read an integer P, representing the position (1-based index) at which the new task must be inserted.
Step 5: Read an integer M, the new task ID to be inserted.
Step 6: If the position is out of range (less than 1 or greater than N+1), print "Position out of range".
Step 7: Otherwise, traverse the list to the correct position and insert the new node.
Step 8: Print the updated linked list.
Step 9: End the program.

Program Code:

```
import java.util.*;

class Node {
   int data;
   Node next;

   Node(int data) {
      this.data = data;
      this.next = null;
   }
}

public class InsertAtPosition {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();

      Node head = null, tail = null;

      // Create the linked list
      for (int i = 0; i < n; i++) {
         int val = sc.nextInt();
         Node newNode = new Node(val);
         if (head == null) {
            head = newNode;
            tail = newNode;
         } else {
            tail.next = newNode;
            tail = newNode;
         }
      }

      int pos = sc.nextInt(); // position (1-based index)
      int newTask = sc.nextInt(); // new task ID

      // Check position validity
      if (pos < 1 || pos > n + 1) {
         System.out.println("Position out of range");
      } else {
```

```
            head = insertAtPosition(head, pos, newTask);
            printList(head);
        }

        sc.close();
    }

    // Insert a new node at a specific position
    static Node insertAtPosition(Node head, int pos, int data) {
        Node newNode = new Node(data);

        // Insert at the beginning
        if (pos == 1) {
            newNode.next = head;
            return newNode;
        }

        Node temp = head;
        for (int i = 1; i < pos - 1 && temp != null; i++) {
            temp = temp.next;
        }

        // If position is valid
        if (temp != null) {
            newNode.next = temp.next;
            temp.next = newNode;
        }

        return head;
    }

    // Print the linked list
    static void printList(Node head) {
        Node temp = head;
        while (temp != null) {
            System.out.print(temp.data + " ");
            temp = temp.next;
        }
    }
}
```

Sample Input:

4
101 102 103 104
2
999

Sample Output:

101 999 102 103 104

Result:

The program successfully inserts a new task at the specified position in the linked list and displays the updated list of task IDs.

3. Expression Converter in a Calculator Application- Using Stack

---

Aim:

To develop a Java program that converts an infix expression into its equivalent postfix expression using stack operations.

Algorithm:

Step 1: Start the program.
Step 2: Read the infix expression as a string input from the user.
Step 3: Initialize an empty stack to store operators and an empty string to store the resulting postfix expression.
Step 4: Traverse each character in the expression:

• If the character is an operand (letter or digit), add it to the result.

• If it is an opening parenthesis '(', push it onto the stack.

• If it is a closing parenthesis ')', pop operators from the stack and append them to the result until '(' is encountered, then pop '('.

• If it is an operator (+, -, *, /), pop all operators from the stack with **greater or equal precedence**, and append them to the result, then push the current operator.

Step 5: After traversing all characters, pop any remaining operators from the stack and append them to the result.

Step 6: Print the final postfix expression.

Step 7: End the program.

Program Code:

```
import java.util.Scanner;
import java.util.Stack;

class Main {
  public static int prec(char c) {
    if (c == '+' || c == '-')
      return 1;
    else if (c == '*' || c == '/')
      return 2;
    else
      return 0;
  }

  public static void main(String args[]) {
    Scanner sc = new Scanner(System.in);
    String s = sc.nextLine().replaceAll("\\s+", "");
    String res = "";
    Stack<Character> stack = new Stack<>();

    for (int i = 0; i < s.length(); i++) {
      char c = s.charAt(i);

      if (Character.isLetterOrDigit(c)) {
        res += c + " ";
      }
      else if (c == '(') {
        stack.push(c);
```

```
    }
    else if (c == ')') {
      while (!stack.isEmpty() && stack.peek() != '(') {
        res += stack.pop() + " ";
      }
      stack.pop();
    }
    else {
      while (!stack.isEmpty() && prec(stack.peek()) >= prec(c)) {
        res += stack.pop() + " ";
      }
      stack.push(c);
    }
   }

   while (!stack.isEmpty()) {
    res += stack.pop() + " ";
   }

   System.out.println(res);
   sc.close();
  }
}
```

Sample Input 1:

A + B * C - D

Sample Output 1:

A B C * + D -

Result:

The program successfully converts an infix expression into its equivalent postfix expression using a stack, ensuring proper operator precedence and parenthesis handling.

4. Ticket Booking System- Using Queue

Aim:

To develop a Java program that implements a queue using an array for managing customer ticket booking requests in a first-come, first-served manner.

Algorithm:

Step 1: Start the program.
Step 2: Create a Queue class with the following attributes:
- queue[] – array to store customer names
- front, rear, size, and capacity – to manage queue operations

Step 3: Implement the following methods in the Queue class:
1. **enqueue(x)** – Adds a customer to the rear of the queue. If the queue is full, display "Queue is full. Cannot add (Customer name)".
2. **dequeue()** – Removes and serves the front customer. If the queue is empty, display "Served Customer: Queue is empty. No customer to serve."
3. **display()** – Prints the current queue elements in order. If empty, print "Queue is empty."

Step 4: In the main() method, read the capacity and number of operations.
Step 5: For each operation:
- If the command is ENQUEUE, add the given customer.
- If the command is DEQUEUE, remove and display the served customer.
- If the command is DISPLAY, print the current queue status.

Step 6: Stop the program.

Program Code:

```
import java.util.Scanner;

class Queue {
  String queue[];
  int f, r, size, cap;

  Queue(int cap) {
```

```
  this.cap = cap;
  queue = new String[cap];
  f = 0;
  r = -1;
  size = 0;
 }

 void enqueue(String x) {
  if (size == cap)
   System.out.println("Queue is full. Cannot add " + x);
  else {
   r = (r + 1) % cap;
   queue[r] = x;
   size++;
  }
 }

 void dequeue() {
  if (size == 0)
   System.out.println("Served Customer: Queue is empty. No customer to serve.");
  else {
   System.out.println("Served Customer: " + queue[f]);
   f = (f + 1) % cap;
   size--;
  }
 }

 void display() {
  if (size == 0)
   System.out.println("Queue is empty.");
  else {
   System.out.print("Current Queue: ");
   for (int i = 0; i < size; i++) {
    int index = (f + i) % cap;
    System.out.print(queue[index] + " ");
   }
   System.out.println();
  }
 }
}
```

```
public class Main {
  public static void main(String args[]) {
    Scanner sc = new Scanner(System.in);
    int cap = sc.nextInt();
    int n = sc.nextInt();
    Queue q = new Queue(cap);

    for (int i = 0; i < n; i++) {
      String s = sc.next();
      if (s.equals("ENQUEUE")) {
        String name = sc.next();
        q.enqueue(name);
      } else if (s.equals("DEQUEUE")) {
        q.dequeue();
      } else if (s.equals("DISPLAY")) {
        q.display();
      }
    }
    sc.close();
  }
}
```

Sample Input:

```
10
5
ENQUEUE Alice
ENQUEUE Bob
DISPLAY
DEQUEUE
DISPLAY
```

Sample Output:

```
Current Queue: Alice Bob
Served Customer: Alice
Current Queue: Bob
```

Result:

The program successfully simulates a ticket booking system using a queue implemented with an array. It correctly handles enqueue, dequeue, and display operations while managing full and empty queue conditions.

5. Tournament Ranking System in Cricket- Using Binary tree

Aim:

To develop a Java program that constructs a binary tree representing team rankings and performs an inorder traversal to display the correct sequence of rankings.

Algorithm:

Step 1: Start the program.
Step 2: Read the number of nodes (teams).
Step 3: Read the list of node values (team rankings) in level order format.
Step 4: Create a binary tree using a queue for level-order insertion.
- The first value is assigned as the root node.
- For each node, insert its left and right children sequentially unless the value is "null".

Step 5: Perform an inorder traversal of the binary tree:
- Traverse the left subtree.
- Visit the root node.
- Traverse the right subtree.

Step 6: Print the node values during the inorder traversal.
Step 7: End the program.

Program Code:

```
import java.util.Scanner;
import java.util.Queue;
import java.util.LinkedList;
```

```
class Node {
  String data;
  Node left, right;
  Node(String ele) {
    this.data = ele;
  }
}

class BT {
  Node root = null;

  void binaryTree(String arr[], int n) {
    root = new Node(arr[0]);
    Queue<Node> q = new LinkedList<>();
    q.add(root);
    int i = 1;

    while (i < n && !q.isEmpty()) {
      Node cur = q.poll();

      if (!arr[i].equals("null")) {
        cur.left = new Node(arr[i]);
        q.add(cur.left);
      }
      i++;

      if (i >= n)
        break;

      if (!arr[i].equals("null")) {
        cur.right = new Node(arr[i]);
        q.add(cur.right);
      }
      i++;
    }
  }

  void inorder(Node root) {
    if (root == null)
```

```
      return;
    inorder(root.left);
    System.out.print(root.data + " ");
    inorder(root.right);
  }
}

public class Main {
  public static void main(String args[]) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    sc.nextLine();
    String arr[] = sc.nextLine().split(" ");
    BT t = new BT();
    t.binaryTree(arr, arr.length);
    t.inorder(t.root);
    sc.close();
  }
}
```

Sample Input:

```
4
1 null 2 3
```

Sample Output:

```
1 3 2
```

Result:

The program successfully constructs a binary tree from level-order input and performs an inorder traversal to display the team rankings in the correct order.

6. Efficient Book Deletion in a Digital Library System

Aim:

To develop a Java program that deletes a specific book (represented by its Book ID) from a Binary Search Tree (BST) while maintaining the BST properties.

Algorithm:

Step 1: Start the program.
Step 2: Read the number of books (n) to be inserted into the BST.
Step 3: Insert each Book ID into the BST using the following logic:
- If the tree is empty, create a new node.
- If the new Book ID is less than the current node's value, insert it into the left subtree.
- If the new Book ID is greater than the current node's value, insert it into the right subtree.

Step 4: Read the Book ID (key) to be deleted.
Step 5: Delete the node using the following cases:
1. **No child (leaf node):** Simply remove the node.
2. **One child:** Replace the node with its child.
3. **Two children:** Replace the node with the inorder successor (minimum value in the right subtree) and delete the successor node.

Step 6: Perform an inorder traversal to print the BST before and after deletion.
Step 7: End the program.

Program Code:

```
import java.util.*;

class Node {
  int data;
  Node left, right;
  Node(int ele) {
    this.data = ele;
    this.left = null;
    this.right = null;
  }
}

class Tree {
```

```
Node BST(int ele, Node root) {
  if (root == null)
    return new Node(ele);
  if (ele < root.data)
    root.left = BST(ele, root.left);
  else if (ele > root.data)
    root.right = BST(ele, root.right);
  return root;
}

void inorder(Node root) {
  if (root == null)
    return;
  inorder(root.left);
  System.out.print(root.data + " ");
  inorder(root.right);
}

Node delete(Node root, int key) {
  if (root == null)
    return null;
  if (key < root.data)
    root.left = delete(root.left, key);
  else if (key > root.data)
    root.right = delete(root.right, key);
  else {
    // Case 1: No child
    if (root.left == null && root.right == null)
      return null;
    // Case 2: One child
    else if (root.left == null)
      return root.right;
    else if (root.right == null)
      return root.left;
    // Case 3: Two children
    Node temp = findMin(root.right);
    root.data = temp.data;
    root.right = delete(root.right, temp.data);
  }
  return root;
```

```
  }

  Node findMin(Node root) {
    while (root.left != null)
      root = root.left;
    return root;
  }
}

public class Main {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    Node root = null;
    Tree t = new Tree();

    // Insert nodes into BST
    for (int i = 0; i < n; i++) {
      int ele = sc.nextInt();
      root = t.BST(ele, root);
    }

    int key = sc.nextInt(); // Book ID to delete

    // Delete the book (node)
    root = t.delete(root, key);

    // Print inorder traversal after deletion
    t.inorder(root);
    sc.close();
  }
}
```

Sample Input:

```
6
50 30 70 20 40 60
30
```

Sample Output:

20 40 50 60 70

Result:

The program successfully deletes a node (Book ID) from the Binary Search Tree while preserving its structural and ordering properties.

7. Ayushi and the Magical Chocolate Piles- Priority Queue

Aim:

To develop a Java program that determines the total number of chocolates remaining after repeatedly selecting the pile with the maximum chocolates and replacing it with its square root value for a given number of iterations.

Algorithm:

Step 1: Start the program.
Step 2: Read the integer N – the number of piles.
Step 3: Read the array of integers representing the number of chocolates in each pile.
Step 4: Read the integer k – the number of times Ayushi can pick chocolates.
Step 5: Use a **max-heap (priority queue)** to efficiently select the pile with the maximum number of chocolates.
Step 6: Repeat the following k times:
- Remove the maximum element (the pile with the most chocolates).
- Replace it with the integer part of its square root (Math.floor(sqrt(value))).
- Add the modified value back to the heap.

Step 7: After all iterations, compute the sum of the elements remaining in the heap.
Step 8: Print the total number of chocolates remaining.
Step 9: Stop the program.

Program Code:

```
import java.util.*;
import java.lang.Math;

public class Main {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

    for (int i = 0; i < n; i++) {
      maxHeap.add(sc.nextInt());
    }

    int k = sc.nextInt();

    for (int i = 0; i < k; i++) {
      int max = maxHeap.poll();           // Get the pile with maximum chocolates
      int newVal = (int) Math.floor(Math.sqrt(max)); // Replace with its square root
      maxHeap.add(newVal);
    }

    int total = 0;
    while (!maxHeap.isEmpty()) {
      total += maxHeap.poll();
    }

    System.out.println(total);
    sc.close();
  }
}
```

Sample Input:

```
5
25 64 9 4 100
```

4

Sample Output:

29

Result:

The program successfully simulates the process of repeatedly selecting the pile with the most chocolates, updating it with its square root, and calculates the total chocolates remaining after the given number of selections.

8. Arranging Students by Marks – Insertion Sort

Aim:

To develop a Java program that sorts a list of students' marks in ascending order using the Insertion Sort algorithm.

Algorithm:

1. Start the program.
2. Read the integer n – the number of students.
3. Read the array of integers representing the students' marks.
4. For each element from index 1 to n-1:
   - Store the current element as key.
   - Compare key with previous elements and shift elements greater than key one position to the right.
   - Place key in its correct position.
5. Repeat until the entire array is sorted.

6. Print the sorted array.

7. Stop the program.

Program Code:

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt();
      int[] marks = new int[n];

      // Read marks
      for (int i = 0; i < n; i++) {
         marks[i] = sc.nextInt();
      }

      // Insertion Sort
      for (int i = 1; i < n; i++) {
         int key = marks[i];
         int j = i - 1;

         while (j >= 0 && marks[j] > key) {
            marks[j + 1] = marks[j];
            j--;
         }
         marks[j + 1] = key;
      }

      // Print sorted marks
      for (int i = 0; i < n; i++) {
         System.out.print(marks[i] + " ");
      }
      sc.close();
   }
```

}

Sample Input :

5
56 12 89 34 22

Sample Output :

12 22 34 56 89

Result:
The program successfully sorts a list of student marks in ascending order using the Insertion Sort algorithm.

9. City Transport Network – Adjacency Matrix Representation

Aim:
To develop a Java program that represents a city transport network using an adjacency matrix. The program determines which cities are directly connected by routes and displays the connections in matrix form.

Algorithm:

1. Start the program.
2. Read the integer n – the number of cities (nodes).
3. Read the integer m – the number of direct routes (edges).
4. Initialize an n × n adjacency matrix with all values as 0.
5. For each of the m direct routes:
    - Read a pair of integers (u, v) representing a direct route from city u to city v.

- Set adjMatrix[u-1][v-1] = 1 to indicate the route.

6. Display the adjacency matrix row by row.

7. Stop the program.

Program Code:

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt(); // Number of cities
      int m = sc.nextInt(); // Number of direct routes

      int[][] adjMatrix = new int[n][n];

      for (int i = 0; i < m; i++) {
         int u = sc.nextInt();
         int v = sc.nextInt();
         adjMatrix[u - 1][v - 1] = 1; // Mark the direct route
      }

      // Print adjacency matrix
      for (int i = 0; i < n; i++) {
         for (int j = 0; j < n; j++) {
            System.out.print(adjMatrix[i][j] + " ");
         }
         System.out.println();
      }

      sc.close();
   }
}
```

Sample Input:

5
6
1 2
1 3
2 4
3 4
4 5
5 1

Sample Output:

0 1 1 0 0
0 0 0 1 0
0 0 0 1 0
0 0 0 0 1
1 0 0 0 0

Result:
The program successfully represents the city transport network using an adjacency matrix, indicating all direct connections between cities.

10.Exploring City Routes – Depth-First Search (DFS) Traversal

Aim:
To develop a Java program that performs a Depth-First Search (DFS) traversal on a graph representing cities and roads, displaying the order in which cities are visited starting from a specified city.

Algorithm:

1. Start the program.
2. Read the integer n – the number of cities (nodes).
3. Read the integer m – the number of roads (edges).
4. Initialize an adjacency list to represent the graph.
5. For each of the m roads:
   • Read a pair of integers (u, v) representing a road between city u and city v.
   • Add v to the adjacency list of u and u to the adjacency list of v (for undirected graph).
6. Read the integer start – the starting city for DFS traversal.
7. Initialize a visited array of size n to keep track of visited cities.
8. Perform DFS recursively:
   • Visit the current city and mark it as visited.
   • Recursively visit all unvisited neighbors of the current city.
9. Print the cities in the order they are visited.
10. Stop the program.

Program Code:

```
import java.util.*;

public class Main {
   static void DFS(int node, ArrayList<Integer>[] graph, boolean[] visited) {
      visited[node] = true;
      System.out.print((node + 1) + " "); // Print city number (1-based)
      for (int neighbor : graph[node]) {
         if (!visited[neighbor]) {
            DFS(neighbor, graph, visited);
         }
      }
```

```
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); // Number of cities
        int m = sc.nextInt(); // Number of roads

        ArrayList<Integer>[] graph = new ArrayList[n];
        for (int i = 0; i < n; i++) {
            graph[i] = new ArrayList<>();
        }

        // Read edges
        for (int i = 0; i < m; i++) {
            int u = sc.nextInt() - 1;
            int v = sc.nextInt() - 1;
            graph[u].add(v);
            graph[v].add(u); // Assuming undirected roads
        }

        int start = sc.nextInt() - 1; // Starting city (0-based)
        boolean[] visited = new boolean[n];

        DFS(start, graph, visited);
        sc.close();
    }
}
```

Sample Input:

```
5
6
1 2
1 3
2 4
3 4
4 5
5 1
3
```

Sample Output:

3 1 2 4 5

Result:

The program successfully performs a DFS traversal on the city transport network and displays the order of cities visited starting from the specified city.